# Laboratório 8 - Dados Relacionais com Polars (IMDb)

Pedro Monteiro Tilhe - RA 254672

14/10/2026

```
import polars as pl
from datetime import datetime

# Saida das tabelas em texto ASCII (compativel com o PDF)
pl.Config.set_tbl_formatting("ASCII_MARKDOWN")
pl.Config.set_tbl_hide_column_data_types(True)
pl.Config.set_tbl_hide_dataframe_shape(True)
pl.Config.set_fmt_str_lengths(40)
pl.Config.set_tbl_rows(20)

PASTA = "../datasets/"

# Os tres arquivos sao TSV (separados por tab) compactados com gzip.
# "\N" representa valor faltante. quote_char=None porque alguns titulos
# contem aspas que nao delimitam campos.
OPCOES = dict(separator="\t", null_values="\\N", quote_char=None)
```

### 1. Leitura dos dados

Lemos apenas as colunas necessárias, para economizar memória.

```
basics = pl.read_csv(
    PASTA + "title.basics0.tsv.gz",
    columns=["tconst", "titleType", "primaryTitle", "startYear", "genres"],
    schema_overrides={"startYear": pl.Int64},
    **OPCOES,
)
```

```
ratings = pl.read_csv(PASTA + "title.ratings.tsv.gz", **OPCOES)

principals = pl.read_csv(
    PASTA + "title.principals0.tsv.gz",
    columns=["tconst", "nconst", "category"],
    **OPCOES,
)

print(basics.shape, ratings.shape, principals.shape)
```

```
(11144942, 5) (1484615, 3) (88359622, 3)
```

Como as perguntas tratam de **filmes**, consideramos apenas os títulos com `titleType == "movie"` que possuem avaliação (junção interna com `ratings` pela chave `tconst`).

```
filmes = (
    basics
    .filter(pl.col("titleType") == "movie")
    .join(ratings, on="tconst", how="inner")
)

filmes.height
```

```
316981
```

### 2. Os 5 filmes com as maiores notas

Ordenamos pela nota média (decrescente) e, em caso de empate, pelo número de votos (decrescente). Como critério final de desempate usamos `tconst`, para o resultado ser determinístico.

```
top5 = (
    filmes
    .sort(
        ["averageRating", "numVotes", "tconst"],
        descending=[True, True, False],
    )
    .select("tconst", "primaryTitle", "startYear", "averageRating", "numVotes")
    .head(5)
)
```

```
top5
```

| tconst | primaryTitle | startYear | averageRating | numVotes |
|---|---|---|---|---|
| ”tt33075815” | ”Kaveri” | 2024 | 10.0 | 1023 |
| ”tt27815015” | ”Kurukku” | 2024 | 10.0 | 451 |
| ”tt28450376” | ”Jedal Dar Omghe 30 Metri” | 2009 | 10.0 | 142 |
| ”tt27859713” | ”Sargashte” | 2016 | 10.0 | 134 |
| ”tt33507675” | ”Gorgeous Rascal” | 2024 | 10.0 | 115 |

Todos os cinco filmes têm nota 10, e o desempate pelo número de votos define a ordem. Note que são filmes com poucos votos; um ranking mais robusto exigiria um número mínimo de votos. Como complemento, o mesmo critério restrito a filmes com pelo menos 100 mil votos:

```
(
    filmes
    .filter(pl.col("numVotes") >= 100_000)
    .sort(
        ["averageRating", "numVotes", "tconst"],
        descending=[True, True, False],
    )
    .select("tconst", "primaryTitle", "startYear", "averageRating", "numVotes")
    .head(5)
)
```

| tconst | primaryTitle | startYear | averageRating | numVotes |
|---|---|---|---|---|
| ”tt0111161” | ”The Shawshank Redemption” | 1994 | 9.3 | 2948622 |
| ”tt0068646” | ”The Godfather” | 1972 | 9.2 | 2055335 |
| ”tt0468569” | ”The Dark Knight” | 2008 | 9.0 | 2929338 |
| ”tt0167260” | ”The Lord of the Rings: The Return of the… | 2003 | 9.0 | 2018311 |
| ”tt0108052” | ”Schindlers List” | 1993 | 9.0 | 1479154 |

### 3. Gênero mais frequente entre os filmes com nota maior que 8

A coluna `genres` pode ter vários gêneros separados por vírgula. Separamos essa coluna em uma lista e usamos `explode` para ter uma linha por par (filme, gênero). Depois contamos os filmes por gênero.

```
generos = (
    filmes
    .filter(pl.col("averageRating") > 8)
    .with_columns(pl.col("genres").str.split(","))
    .explode("genres")
    .drop_nulls("genres")
    .group_by("genres")
    .agg(pl.len().alias("n_filmes"))
    .sort(["n_filmes", "genres"], descending=[True, False])
)

generos.head(5)
```

| genres | n_filmes |
|---|---|
| ”Documentary” | 10564 |
| ”Drama” | 8114 |
| ”Comedy” | 3186 |
| ”Biography” | 1395 |
| ”Romance” | 1363 |

```
genero_top = generos.row(0, named=True)
print(f"Genero mais frequente: {genero_top['genres']} "
      f"({genero_top['n_filmes']} filmes)")
```

```
Genero mais frequente: Documentary (10564 filmes)
```

### 4. Os 3 atores/atrizes com mais filmes de nota maior que 7.5

Selecionamos em `principals` as categorias `actor` e `actress`, juntamos com os filmes de nota maior que 7.5 e contamos o número de filmes distintos por pessoa (`nconst`). Os arquivos fornecidos não têm o nome das pessoas, então elas são identificadas pelo `nconst`.

```
filmes_bons = filmes.filter(pl.col("averageRating") > 7.5).select("tconst")

atores = (
    principals
    .filter(pl.col("category").is_in(["actor", "actress"]))
    .join(filmes_bons, on="tconst", how="inner")
    .group_by("nconst")
```

```
    .agg(
        pl.col("tconst").n_unique().alias("n_filmes"),
        pl.col("category").first().alias("categoria"),
    )
    .sort(["n_filmes", "nconst"], descending=[True, False])
)

atores.head(3)
```

| nconst | n_filmes | categoria |
|---|---|---|
| ”nm0004660” | 199 | ”actor” |
| ”nm0595934” | 155 | ”actor” |
| ”nm3183374” | 124 | ”actor” |

### Data e hora de compilação

14/10/2026 10:57:13